Veri Yapıları Lab 7

Kuyruklar (Queues)

- Kuyruk, elemanların bir uçtan (**rear**) eklendiği ve diğer uçtan (**front**) silindiği doğrusal bir listedir. İlk giren ilk çıkar: **FIFO** (First In First Out). Banka gişesindeki sıra en bilinen örnektir.

İşlem	Ne yapar	Boş kuyrukta
`enqueue(q, x)`	x'i sona (rear) ekler	normal çalışır
`dequeue(q, &x)`	baştaki elemanı siler, x'e verir	`false` (underflow)
`queueFront(q, &x)`	baştaki elemanı okur, silmez	`false`
`queueRear(q, &x)`	sondaki elemanı okur, silmez	`false`
`emptyQueue(q)`	boş mu? (`count == 0`)	`true`
`queueCount(q)`	eleman sayısı	`0`

Kuyruklar (Queues)

Bağlı liste ile gerçekleştirim. Kuyruk yapısı (QUEUE) elemanları değil, kuyruğun bilgisini tutar: ilk düğüm (`front`), son düğüm (`rear`) ve eleman sayısı (`count`). Elemanlar düğümlerde saklanır; her düğüm veriyi (`void* dataPtr`) ve bir sonraki düğümün adresini (`next`) tutar. Normalde ekleme yalnızca `rear`'ı, silme yalnızca `front`'u değiştirir. İki özel durumda ikisi birlikte değişir:

- Boş kuyruğa ekleme: tek düğüm hem ilk hem son eleman olduğu için `front` ve `rear` yeni düğümü gösterir.
- Tek elemanlı kuyruktan silme: kuyruk boşalır, `front` ve `rear` NULL olur.

```
bool enqueue(QUEUE* q, void* itemPtr)
{
    QUEUE_NODE* newPtr = malloc(sizeof(QUEUE_NODE));
    if (!newPtr) return false;          /* overflow */
    newPtr->dataPtr = itemPtr;
    newPtr->next    = NULL;
    if (q->count == 0)                  /* boş kuyruk */
        q->front = newPtr;
    else
        q->rear->next = newPtr;
    q->rear = newPtr;
    q->count++;
    return true;
}
```

Kuyruklar (Queues)

- **dequeue ve queueFront.** `dequeue` baştaki düğümü siler ve verisini `itemPtr` ile çağırana verir; `queueFront` yalnızca okur. Boş kuyrukta ikisi de `false` döner (underflow).

```
bool dequeue(QUEUE* q, void** itemPtr)
{
    QUEUE_NODE* del;
    if (!q->count) return false; /* underflow */
    *itemPtr = q->front->dataPtr;
    del = q->front;
    if (q->count == 1)           /* tek eleman */
        q->front = q->rear = NULL;
    else
        q->front = q->front->next;
    q->count--;
    free(del);
    return true;
}
```

```
bool queueFront(QUEUE* q, void** itemPtr)
{
    if (!q->count) return false;
    *itemPtr = q->front->dataPtr;
    return true;                 /* silmez */
}

/* Kullanım */
char* s;
if (dequeue(q, (void*)&s))
    printf("%s\n", s);
```

Kuyruklar (Queues)

- **Dizi ile gerçekleştirim (dairesel kuyruk).** Silme yapıldıkça dizinin başı boşalır; `rear` sona gelince yer varken kuyruk "dolu" görünür (Figure 5-16). Çözüm, indisleri `% MAX` ile başa döndürmektir: `rear = (rear + 1) % MAX`, `front = (front + 1) % MAX`.
- **Önemli not.** Kuyrukta gezinme (traversal) yoktur. İçeriği bozmadan okumak için `count` kez `dequeue` edip aynı elemanı geri `enqueue` ederiz (kuyruğu döndürme). `dequeue` çağrılırken veri pointer'ı `void**`'a uyması için cast edilir: `dequeue(q, (void*)&dataPtr)`.

```
/* Dizi ile dairesel kuyruk */
bool enqueue(AQUEUE* q, int x) {
    if (q->count == MAX) return false;
    q->rear = (q->rear + 1) % MAX;
    q->data[q->rear] = x;
    q->count++;  return true;
}
bool dequeue(AQUEUE* q, int* x) {
    if (q->count == 0) return false;
    *x = q->data[q->front];
    q->front = (q->front + 1) % MAX;
    q->count--;  return true;
}
```

```
/* Kuyruğu bozmadan yazdırma */
int* p;
int n = queueCount(q);
for (int i = 0; i < n; i++) {
    dequeue(q, (void*)&p);    /* cast */
    printf("%d ", *p);
    enqueue(q, p);            /* geri koy */
}
```

Örnek 1: Bağlı liste ile kuyruk izleme

```
/* Boş kuyrukla başla */
 1. enqueue(4)
 2. enqueue(7)
 3. dequeue
 4. dequeue
 5. enqueue(2)
 6. enqueue(6)
 7. enqueue(9)
 8. queueFront
 9. dequeue
10. queueRear
```

Boş bir kuyrukla başlayıp soldaki işlemleri sırayla yapan tam C programını yazın. `dequeue`, `queueFront` ve `queueRear` ile alınan değerleri sırayla ekrana yazdırın, en sonda `count` değerini yazdırın.

Beklenen çıktı

```
4 7 2 2 9
count = 2
```

Kuyruk takibi

```
4
4 7
7
null
2
2 6
2 6 9
6 9
```

Örnek 2: Dairesel dizi kuyruğu

```
#define MAX 6
front = 0, rear = -1, count = 0

enqueue(3)  enqueue(5)  enqueue(7)
enqueue(9)  enqueue(11)
dequeue  dequeue  dequeue
enqueue(13) enqueue(15) enqueue(17)
dequeue
```

Dairesel dizi kuyruğu ile soldaki işlemleri yapan tam C programını yazın. Ekleme `rear = (rear + 1) % MAX`, silme `front = (front + 1) % MAX` ile yapılır. Program; çıkan değerleri, son `front`/`rear`/`count` değerlerini, kuyruğun içeriğini ve 3 ekleme daha denendiğinde hangisinin başarısız olduğunu yazmalı.

Beklenen çıktı

```
3 5 7 9
front=4 rear=1 count=4
11 13 15 17
3. ekleme basarisiz
```

Örnek 3: Negatif sayıları kuyruktan silme

Bağlı liste kuyruğunda `int*` veriler tutuluyor. Negatif sayıları kuyruktan silen, diğerlerinin sırasını koruyan `removeNegatives` fonksiyonunu ve onu `5 -3 8 -1 0 7` verisiyle test eden `main`'i içeren tam C programını yazın. Yalnızca ADT fonksiyonlarını kullanın; `front->next` gibi doğrudan erişim yasaktır.

```
void removeNegatives(QUEUE* q)
{
    /* Sadece ADT fonksiyonları: */
    /* enqueue, dequeue, queueCount */
}
```

Beklenen çıktı

```
Once : 5 -3 8 -1 0 7
Sonra: 5 8 0 7
count = 4
```

İpucu: Kuyrukta gezinme yoktur; kuyruğu döndürün.